



深圳職業技術大學

SHENZHEN POLYTECHNIC UNIVERSITY

数据结构课程

二叉树遍历实验项目

实验实训报告

一、实验实训项目名称

二叉树遍历实验

二、项目内容与要求

实验目的

1. 理解二叉树的逻辑结构及二叉链表存储方式。
2. 掌握由先序扩展序列建立二叉树的方法。
3. 熟悉递归思想在树结构处理中的应用。
4. 掌握二叉树先序遍历、中序遍历、后序遍历的实现过程。
5. 提高使用 C++ 完成数据结构实验的能力，特别是指针、动态内存和函数设计的综合运用能力。

实验要求

- 1.使用 C++ 完成程序设计与调试。
- 2.输入一组按先序方式给出的二叉树扩展序列，其中字符 0 表示空结点。
- 3.将该序列还原为二叉链表形式的二叉树。
- 4.在建立好的二叉树上分别完成先序、中序和后序遍历。
- 5.输出三种遍历结果，每种结果各占一行。
- 6.程序应保证逻辑清晰、边界处理正确，并能正确释放动态申请的结点空间。

实验内容

本实验的核心任务是“把先序扩展序列”转换为“二叉链表”，然后在此基础上完成三种基本遍历。

已知输入的结点序列遵循如下规则：

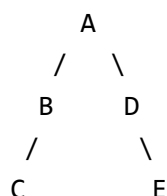
- 按先序顺序给出结点值。

- 若某位置为空地子树，则用字符 0 占位。

例如序列：

A B C 0 0 0 D 0 E 0 0

表示的二叉树结构为：



根据该树进行遍历，可得到：

- 先序遍历：ABCDE
- 中序遍历：CBADE
- 后序遍历：CBEDA

实验原理

1. 二叉链表存储原理

二叉树的链式存储通常采用二叉链表。每个结点包含三个部分：

- 数据域 data，用于保存当前结点的信息。
- 左孩子指针 lchild。
- 右孩子指针 rchild。

这种结构能够直观表示任意形态的二叉树，适合递归建立与递归遍历。

2. 先序扩展序列建树原理

本实验输入的并不是普通先序序列，而是带空结点标记的先序扩展序列。它的关键价值在于：

仅凭这一组序列，就可以唯一确定一棵二叉树。

建树时按如下思路递归处理：

1. 读取当前位置字符。

- 2.若该字符为 θ ，说明当前位置为空，返回空指针。
- 3.若该字符为有效结点，则创建新结点。
- 4.继续递归建立该结点的左子树。
- 5.再递归建立该结点的右子树。

由于输入本身就是“根 $\rightarrow$ 左 $\rightarrow$ 右”的顺序，所以递归过程与先序定义天然一致。

3. 三种遍历原理

- 先序遍历：先访问根结点，再访问左子树，最后访问右子树。
- 中序遍历：先访问左子树，再访问根结点，最后访问右子树。
- 后序遍历：先访问左子树，再访问右子树，最后访问根结点。

三种遍历的区别，本质上是“访问根结点”的时机不同。

4. 算法复杂度分析

设扩展序列长度为 n 。

- 建树过程对每个输入元素只处理一次，时间复杂度为 $O(n)$ 。
- 三种遍历都需要访问每个有效结点一次，时间复杂度均为 $O(n)$ 。
- 递归调用所需的额外空间与树高有关，空间复杂度为 $O(h)$ 。

三、报告内容

实验步骤

1. 定义二叉树结点结构

建立 `BiNode` 结构体，包含数据域和左右孩子指针，用于描述二叉链表中的一个结点。

2. 读入实验数据

先输入扩展序列长度 `n`，再依次读入 `n` 个字符，并存入顺序容器中，便于后续按位置递归处理。

3. 递归建立二叉树

设置位置变量 `pos` 记录当前处理到序列中的第几个元素。

调用 `CreateTree` 函数：

- 遇到 `0` 时返回空指针。
- 遇到有效字符时创建结点。
- 递归建立左子树和右子树。

4. 实现三种递归遍历

分别编写：

- `PreOrder`
- `InOrder`
- `PostOrder`

三个函数，按对应遍历次序输出结点值。

5. 输出结果并释放空间

依次输出先序、中序和后序遍历结果。

最后调用 `DestroyTree` 后序释放整棵树，避免内存泄漏。

实验代码

```
struct BiNode
{
    char data;
    BiNode *lchild;
    BiNode *rchild;
};

// 根据先序序列建立二叉树, 字符'0'表示空结点
BiNode *CreateTree(const vector<char> &sequence, int &pos)
{
    if (pos >= sequence.size())
    {
        return nullptr;
    }

    if (sequence[pos] == '0')
    {
        ++pos;
        return nullptr;
    }

    BiNode *root = new BiNode;
    root->data = sequence[pos++];
    root->lchild = CreateTree(sequence, pos);
    root->rchild = CreateTree(sequence, pos);
    return root;
}
```

```
// 先序遍历
void PreOrder(BiNode *root)
{
    if (root == nullptr)
    {
        return;
    }
    cout << root->data;
    PreOrder(root->lchild);
    PreOrder(root->rchild);
}
```

```
// 中序遍历
void InOrder(BiNode *root)
{
    if (root == nullptr)
    {
        return;
    }
    InOrder(root->lchild);
    cout << root->data;
    InOrder(root->rchild);
}
```

```
// 后序遍历
void PostOrder(BiNode *root)
{
    if (root == nullptr)
    {
        return;
    }
    PostOrder(root->lchild);
    PostOrder(root->rchild);
    cout << root->data;
}
```

```
// 释放二叉树空间
void DestroyTree(BiNode *root)
{
    if (root == nullptr)
    {
        return;
    }
    DestroyTree(root->lchild);
    DestroyTree(root->rchild);
    delete root;
}
```

```
int main()
{
    int n;
    cin >> n;

    vector<char> sequence(n);
    for (int i = 0; i < n; ++i)
    {
        cin >> sequence[i];
    }

    int pos = 0;
    BiNode *tree = CreateTree(sequence, pos);

    PreOrder(tree);
    cout << endl;
    InOrder(tree);
    cout << endl;
    PostOrder(tree);
    cout << endl;

    DestroyTree(tree);
    return 0;
}
```

输入输出：

```
11
A B C 0 0 0 D 0 E 0 0
ABCDE
CBADE
CBEDA
```

3、实验结论及思考：

从输出结果可以看出，程序已经正确完成了二叉树的建立和三种遍历。

先序结果 **ABCDE** 说明程序能够按照“根、左、右”的顺序访问所有有效结点；

中序结果 **CBADE** 说明左子树访问完成后才访问根结点，遍历次序符合中序定义；

后序结果 **CBEDA** 则说明程序先处理左右子树，最后访问根结点，结果与理论分析一致。

与实验 PDF 中给出的基础写法相比，本次代码实现还有以下改进：

1.未使用全局变量记录建树位置，而是通过引用参数传递当前位置，函数独

立性更好。

2. 使用 `vector<char>` 保存输入序列，避免固定数组长度不足或下标越界问题。
3. 增加 `DestroyTree` 函数释放动态内存，使程序结构更完整。
4. 函数命名更加清晰，便于阅读、调试和后续扩展。

本实验说明，递归特别适合处理树这种天然分层的数据结构。建树过程与遍历过程都可以拆分为“处理当前结点 + 递归处理子树”的形式，因此代码简洁、逻辑清楚。但与此同时，递归程序也必须严格处理边界条件，特别是空结点和序列结束位置，否则容易出现访问越界或无限递归等错误。

总体来看，本次实验达到了预期目标，不仅实现了二叉树的三种基本遍历，也进一步加深了对“树的链式存储 + 递归算法”这一核心思想的理解。